

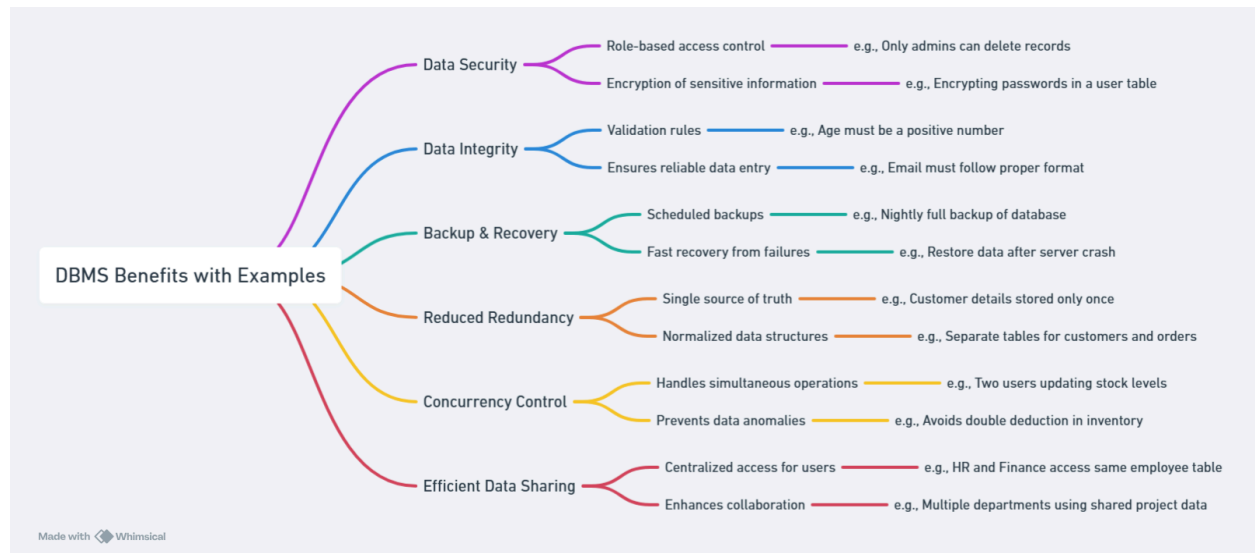
1.Comparison between Flat file & relationship database :

Criteria	Flat File System	Relational Database
Structure	Data is stored in a single table or plain text file (e.g., CSV, TXT).	Data is stored in multiple related tables using rows and columns with defined schemas.
Data Redundancy	High – the same data might be repeated in multiple places.	Low – normalization reduces duplication by organizing data into related tables.
Relationships	No inherent support for relationships between data entries.	Supports relationships (one-to-one, one-to-many, many-to-many) using keys.
Example Usage	Simple applications, small-scale tools, data import/export, log files.	Business applications, websites, inventory systems, financial and customer data.
Drawbacks	- Hard to maintain- Prone to redundancy- Poor scalability and query support	- Requires database management systems- Complexity in setup and maintenance

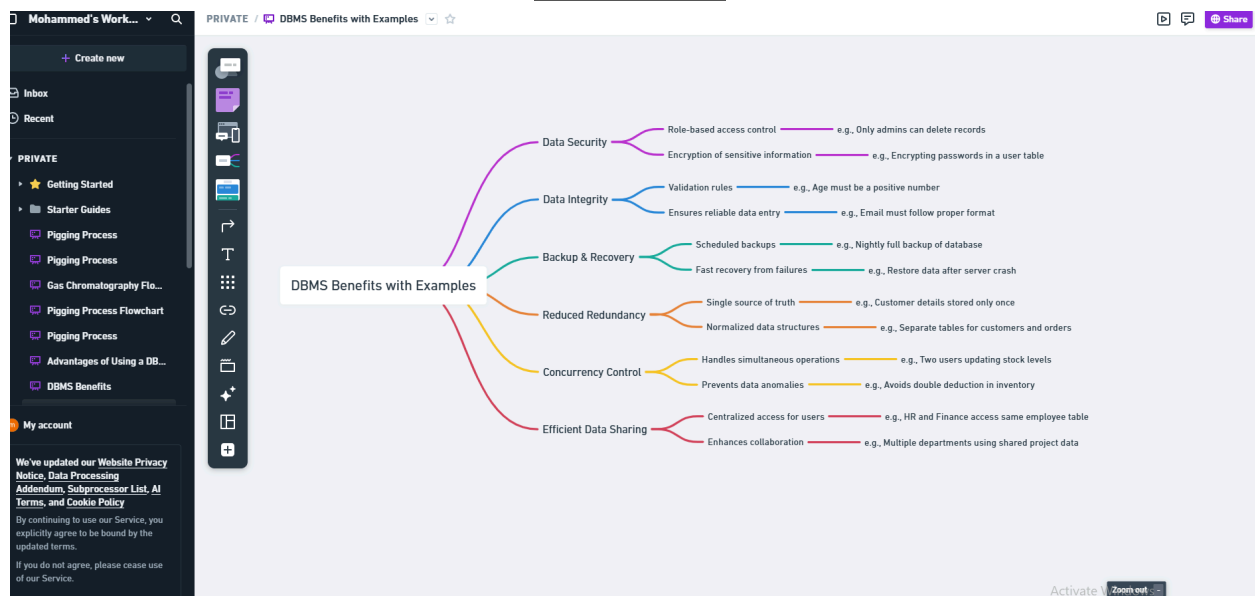
2. DBMS Advantages Mind Map :

Reference :

<https://whimsical.com/advantages-of-using-a-dbms-5sMumcQLAm2qJQ9YAS1NtE>



PROGRESS :-





Drawing - Tool : whimsical.com

1. Security 🔒

What it means:

DBMS allows access control through authentication (usernames/passwords) and permissions (read/write).

Example:

In a hospital database, only doctors can view patient medical records, while receptionists can only see appointment schedules. Unauthorized users can't access sensitive data.

2. Integrity ✓

What it means:

Data integrity ensures that the data stored in the database is accurate and consistent, using rules like primary keys and data types.

Example:

A student's grade must be between 0 and 100. If someone tries to enter a grade of 150, the DBMS will reject it due to integrity constraints.

3. Backup

What it means:

A DBMS can create automatic or scheduled backups so data can be restored if lost.

Example:

An online store crashes due to a server failure. The DBMS restores all product listings and customer orders from the last backup with minimal downtime.

4. Redundancy

What it means:

Redundancy refers to storing the same data multiple times. DBMS reduces it using normalization.

Example:

Instead of writing a customer's name every time they place an order, DBMS uses a CustomerID. This saves space and keeps the data consistent if the customer's info changes.

5. Concurrency

What it means:

Multiple users can use the database at the same time without causing errors or data conflicts.

Example:

At a bank, one teller updates a customer's balance after a deposit, while another accesses the same customer's loan details—both actions happen smoothly without conflict.

6. Data Sharing

What it means:

A DBMS enables data to be shared among multiple users or applications securely.

Example:

In a school management system, teachers, students, and administrators access the same database for grades, attendance, and fees—but see only what they're allowed to.

1. System Analyst

Role:

Acts as a bridge between business needs and technical teams.

Responsibilities:

- Understands user requirements
- Designs system processes and data flow
- Suggests improvements

Example:

Before building a school database, a system analyst interviews staff to understand what features (e.g., attendance tracking, grade reports) are needed.

2. Database Designer

Role:

Designs the structure and schema of the database.

Responsibilities:

- Defines tables, fields, keys, and relationships
- Ensures normalization to avoid redundancy
- Creates ER diagrams

Example:

For a library system, the designer sets up tables like Books, Authors, and Borrowers, with correct primary/foreign keys.

3. Database Developer

Role:

Implements the database and writes the backend logic.

Responsibilities:

- Writes SQL queries, stored procedures, triggers
- Populates the database
- Optimizes query performance

Example:

In an e-commerce app, the developer writes a stored procedure to automatically calculate discounts on checkout.

4. DBA (Database Administrator)

Role:

Manages and maintains the database after it is deployed.

Responsibilities:

- Handles security, backups, and recovery
- Tunes performance
- Manages user roles and permissions

Example:

If a database slows down, the DBA investigates and optimizes indexes or queries to fix it.

5. Application Developer

Role:

Builds the frontend or business-layer applications that use the database.

Responsibilities:

- Connects applications to the database using APIs or ORMs
- Handles form input and displays results from the database

Example:

In a hotel booking system, they create the web interface where users search rooms, and that data is fetched from the database.

6. BI (Business Intelligence) Developer

Role:

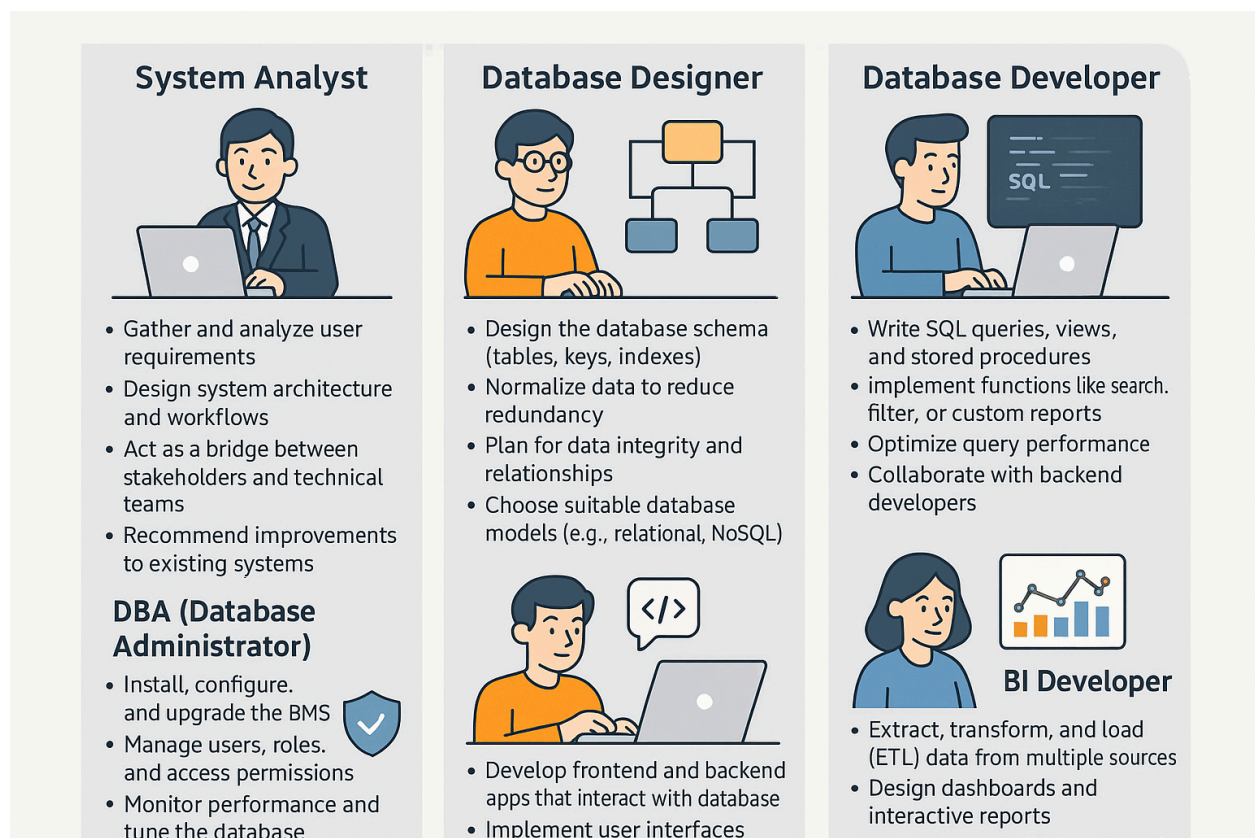
Uses database data to build dashboards and reports for decision-making.

Responsibilities:

- Creates ETL pipelines (Extract, Transform, Load)
- Builds visualizations using tools like Power BI or Tableau
- Helps businesses understand trends

Example:

A BI developer builds a sales dashboard showing weekly revenue trends across regions from the company's database.



Additional Research Topics to Include in the Report:

Types of Databases:

1. Relational vs Non-Relational

- **Relational Databases:** Store data in tables with rows and columns. Data is structured and relationships are defined using keys.
 - **Examples:** MySQL, PostgreSQL, Oracle
 - **Use Case:** Banking systems, HR systems
- **Non-Relational Databases:** Store data in formats like key-value pairs, documents, wide-columns, or graphs. Designed for scalability and flexibility.
 - **Examples:** MongoDB (Document), Cassandra (Wide-Column)
 - **Use Case:** Real-time analytics, content management, IoT

2. Centralized vs Distributed vs Cloud Databases

- **Centralized Database:** Data is stored and maintained in a single location.
 - **Use Case:** Small businesses with minimal data
- **Distributed Database:** Data is spread across multiple locations but appears as one logical database.
 - **Use Case:** Global applications requiring low latency and high availability
- **Cloud Database:** Hosted and accessed via cloud services like AWS, Azure, or Google Cloud.
 - **Use Case:** Scalable applications with global access and high uptime requirements

Cloud Storage and Databases

- **Cloud Storage:** A service model in which data is stored, managed, and backed up remotely and made available over the internet.
 - **Relation to Databases:** Cloud databases use cloud storage as their foundation, benefiting from scalability, managed services, and backup.

Advantages of Cloud Databases:

- Automatic backups and updates
- High availability and scalability
- Reduced infrastructure management

Disadvantages:

- Potential security concerns
- Dependence on internet connectivity
- Subscription-based cost may increase over time

Examples: Azure SQL Database, Amazon RDS, Google Cloud Spanner

Database Engines and Languages

- **Database Engine:** The core service for storing, processing, and securing data. It provides access and control through a query language.

Examples and Their Languages:

- **SQL Server:** T-SQL
- **MySQL:** ANSI SQL, MySQL dialect
- **Oracle:** PL/SQL
- **PostgreSQL:** ANSI SQL, PL/pgSQL

Engine-Language Relationship: Each engine has a primary query language, often with proprietary extensions.

Cross-Engine Language Use: Basic SQL can work across engines, but advanced queries and stored procedures may not.

Can We Transfer a Database Between Engines?

Yes, but it involves challenges:

- Data type differences
- Syntax and language incompatibility (T-SQL vs PL/SQL)
- Stored procedures, functions, triggers need rewriting

Considerations Before Migration:

- Compatibility of data types and constraints
 - Rebuilding of indexes, triggers, and relationships
 - Testing and data integrity validation post-transfer
-

Logical vs Physical Schema

- **Logical Schema:** Abstract design that outlines tables, relationships, and constraints without concern for physical implementation.
- **Physical Schema:** Actual implementation of the database on a storage system with specifics like indexes, partitions, and storage format.

Differences:

- **Logical:** What data is stored
- **Physical:** How data is stored

Importance:

- Helps in optimizing performance, scaling, and maintenance

Example: Student Entity

- **Logical Schema:**
 - Student(ID, Name, Email)
 - **Physical Schema:**
 - Table: Students
 - Fields: ID INT PRIMARY KEY, Name VARCHAR(100), Email VARCHAR(100)
 - Index on Email, Stored in SSD partition A
-